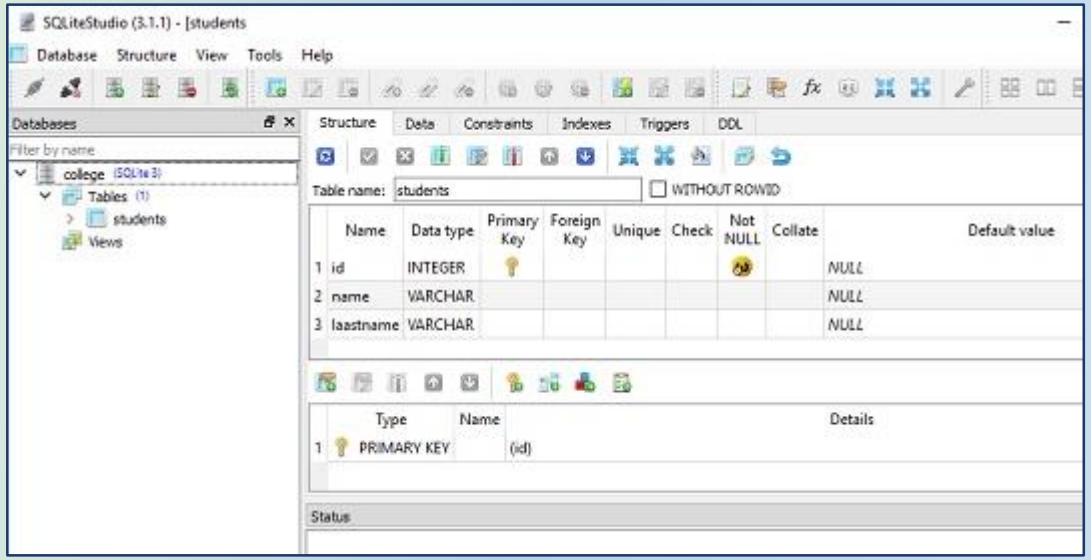
Работа с БД. Графический интерфейс. Web-frameworks (обзорно)

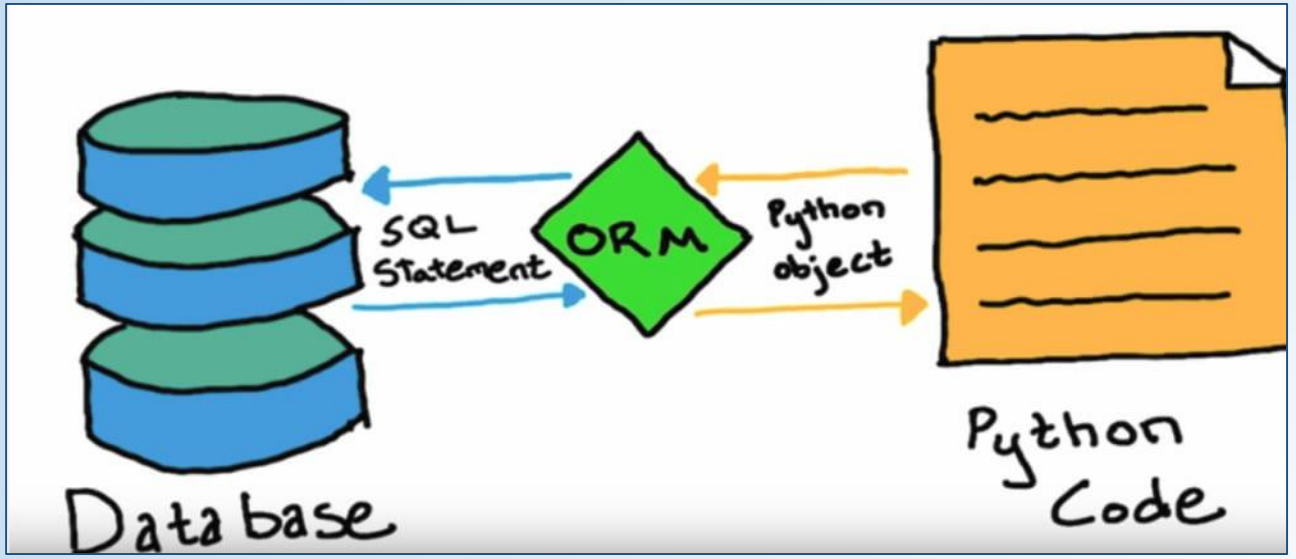
Лекция 8

Реляционные СУБД и Python

Для работы с реляционной СУБД в Python можно использовать два подхода:

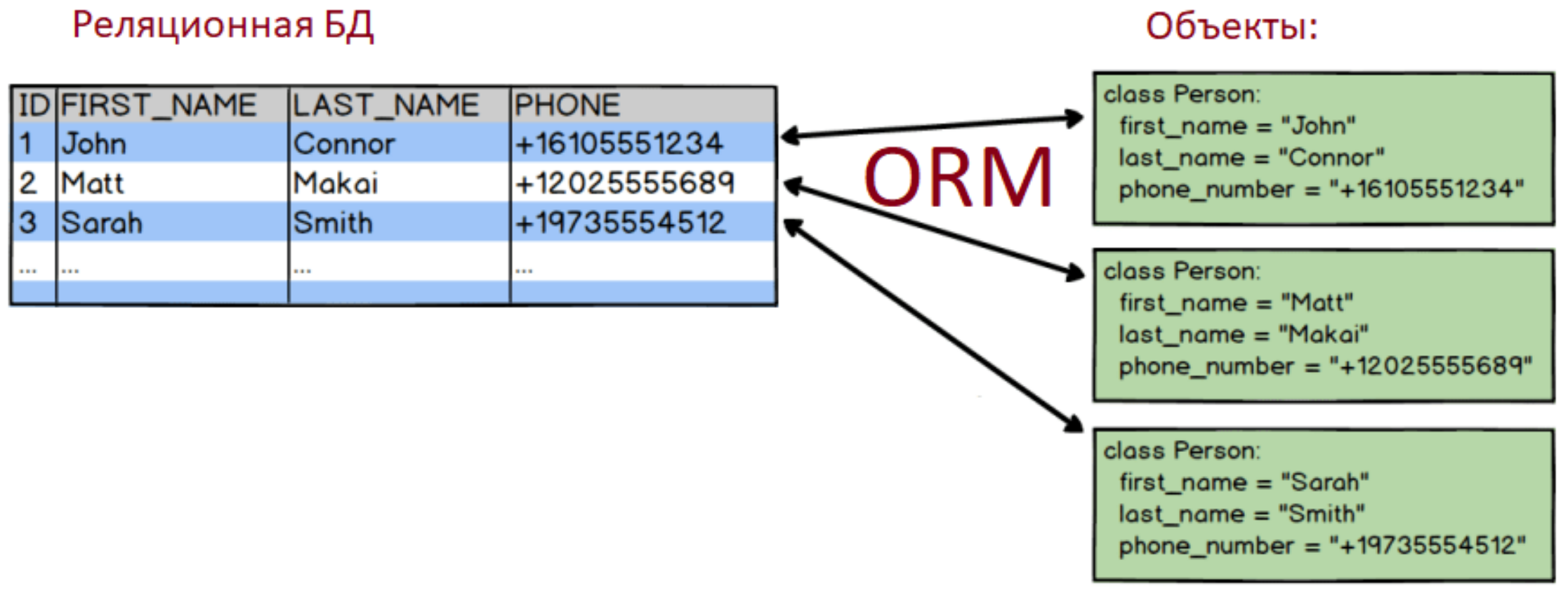
1. Воспользоваться библиотекой, которая соответствует конкретной СУБД, и использовать для работы с БД язык SQL. Например, для работы с SQLite используется модуль sqlite3 (этот модуль встроен в стандарт):
2. Воспользоваться ORM (техника программирования, предназначенная для отображения данных объектно-ориентированного языка программирования на хранилище, чаще всего на реляционную СУБД), которая использует объектно-ориентированный подход для работы с БД. Например, SQLAlchemy.





ORM: Object Relational Mapping

ORM предоставляет инструментарий для превращения записей базы данных в объекты, делая взаимодействие с базой данных как объектами в языках ООПрограммирования.



ORM: Object Relational Mapping

Active Record – основной шаблон проектирования ORM, согласно которому каждая таблица в базе данных заключена в модель.

Каждый объект этой модели будет жестко привязан к записи в базе данных.

Используя этот шаблон, не нужно настраивать ни свойства ни связи с базой данных (объекты это получают автоматически согласно схеме БД).

Active Record имеет необходимые встроенные методы: создание-чтение-обновление-удаление для манипулирования объектами.

Шаблон Active Record использует фреймворк Django.

Шаблон Active Record создаёт модель для БД:

```
Base = declarative_base()

class Person(Base):
    __tablename__ = 'Person'

    id = Column(Integer, primary_key=True)
    first_name = Column(String)
    last_name = Column(String)
    address = Column(String)
```

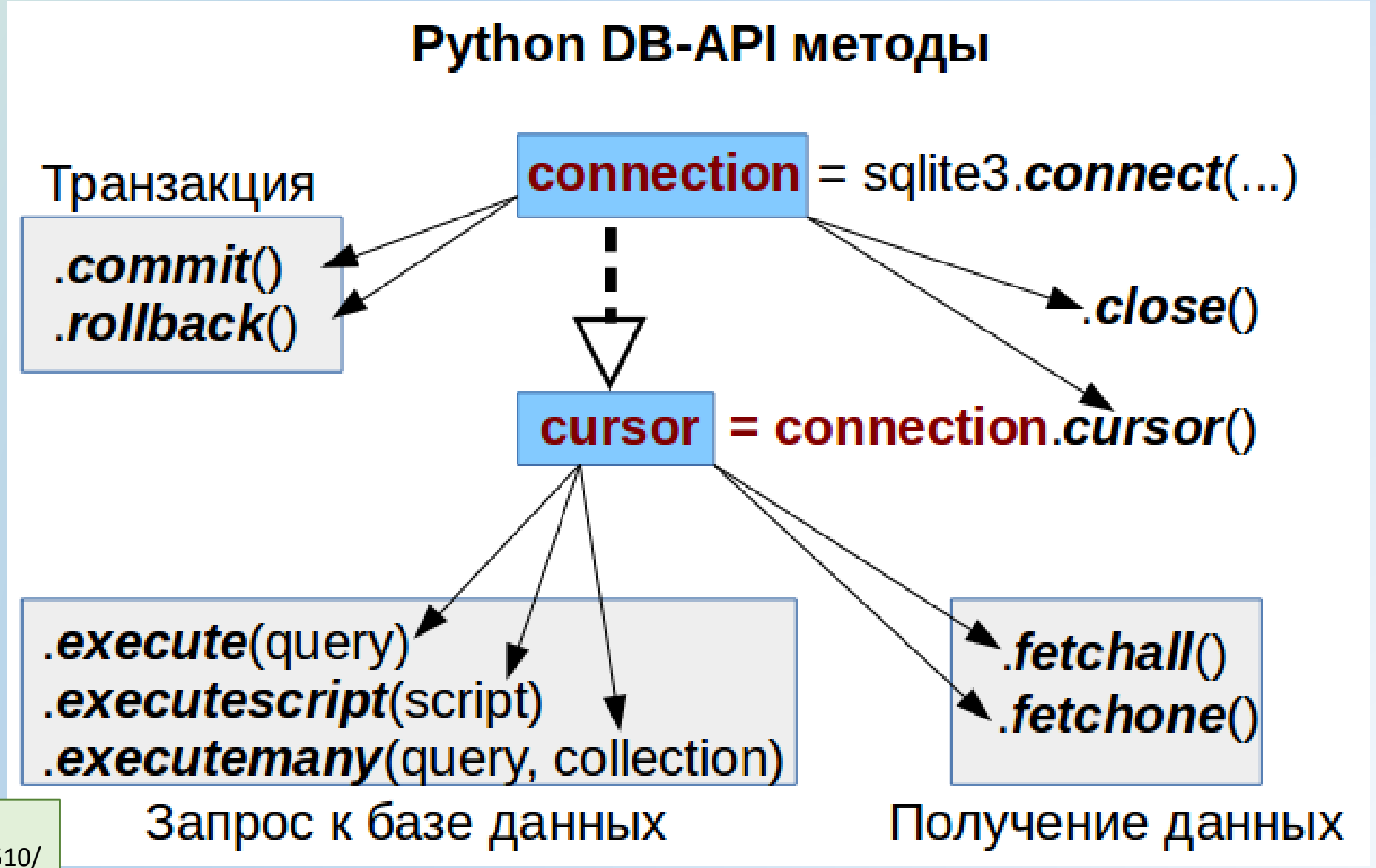
Библиотека для конкретной СУБД

Python поддерживает работу с большим количеством СУБД.
Схема взаимодействия с большинством СУБД:

1. Импорт библиотеки (DB-API модуля), соответствующую типу базы данных
2. Создание соединения с базой данных
3. Создание курсора – это специальный объект, который делает запросы и получает их результаты
4. Выполнение запросов к БД и получение данных….
5. Закрытие соединения с базой данных

СУБД	API модуль (DB-API)
SQLite	Sqlite3
PostgreSQL	Psycopg2
MySQL	mysql.connector
ODBC	pyodbc

Методы взаимодействия с БД (модуль SQLite)

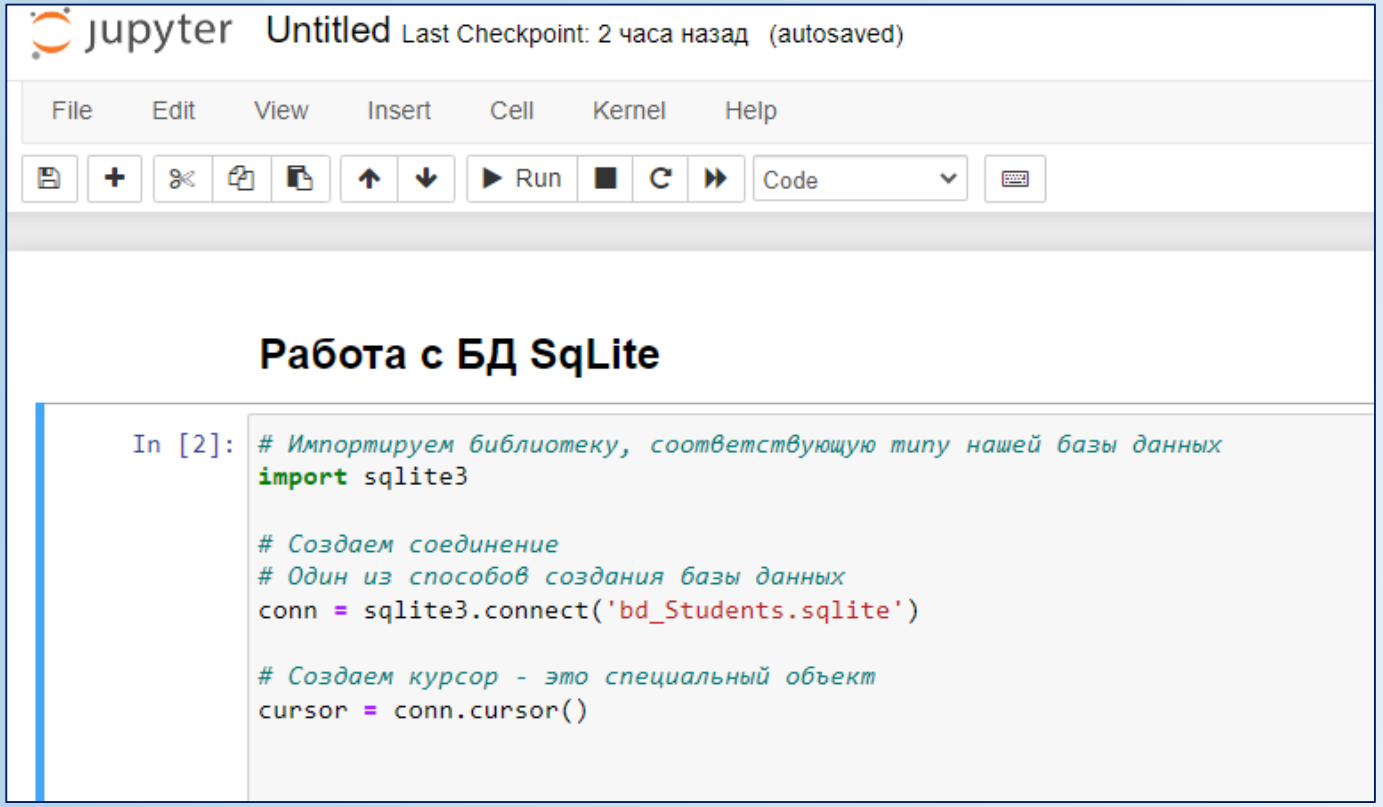


Источник:
https://habr.com/ru/post/321510/

Методы взаимодействия с БД (модуль SQLite)

Основные типы SQLite (мало, но в этом есть и свои плюсы):

- NULL -- значение NULL
- INTEGER -- целое число
- REAL-- число с плавающей точкой
- TEXT -- текст
- BLOB -- бинарное представление крупных объектов (сохраняются в том же виде, как их ввели)



Методы взаимодействия с БД (модуль SQLite)

In [11]:
```
# создаём таблицу в БД
cursor.execute('CREATE TABLE IF NOT EXISTS students(userid INTEGER PRIMARY KEY, name TEXT, balls REAL, info BLOB)')
```

Out[11]:
```
<sqlite3.Cursor at 0x20818fa2490>
```

In [12]:
```
#закрепляем изменения
conn.commit()
```

In [13]:
```
# добавляем строки в таблицу (3 строки)
studs = []
studs.append(('1', 'Иванов', 56.89, '345 3893 fdjf'))
studs.append(('3', 'Петров', 46.89, '345 3893 fdjf'))
studs.append(('4', 'Сидоров', 67.89, '345 3893 fdjf  ooo'))
```

In [18]:
```
for x in studs:
    cursor.execute('INSERT INTO students VALUES(?, ?, ?, ?)', x)
```

In [ ]:
```
#закрепляем изменения
conn.commit()
```

Методы взаимодействия с БД (модуль SQLite)

In [24]:
```
# выполняем запрос по извлечению данных из таблицы БД
cursor.execute("SELECT * FROM students")
# получаем очередную строку по рузультату
result = cursor.fetchone()
print(result)
# получаем очередную строку по рузультату
result = cursor.fetchone()
print(result)
# можно организавать бесконечный цикл
```

```
(1, 'Иванов', 56.89, '345 3893 fdjf')
(3, 'Петров', 46.89, '345 3893 fdjf')
```

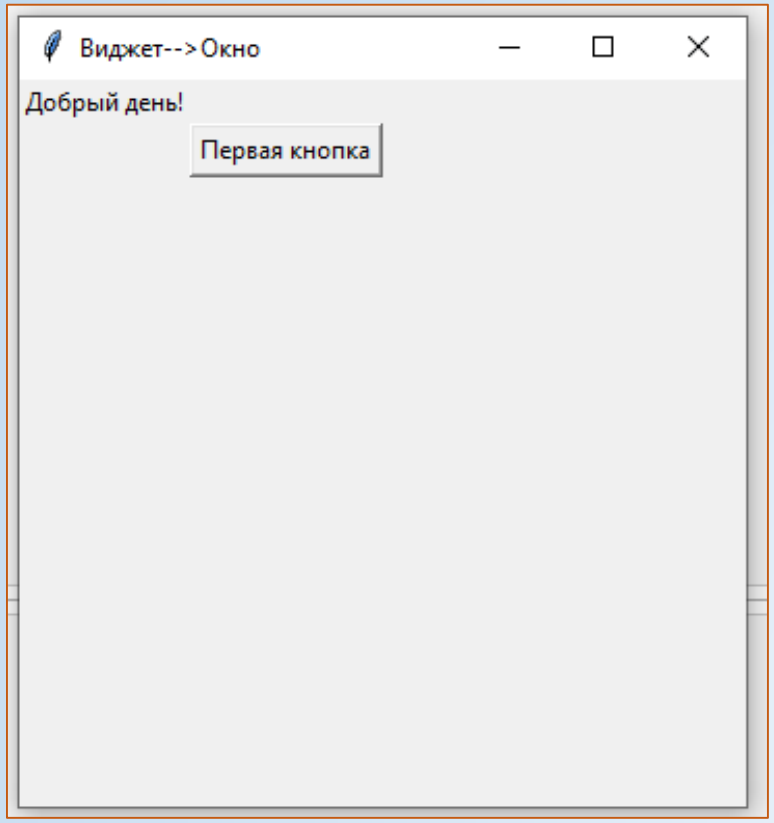
In [23]:
```
# получаем весь результат
cursor.execute("SELECT * FROM students")
results = cursor.fetchall()
print(*results, sep='\n')
```

```
(1, 'Иванов', 56.89, '345 3893 fdjf')
(3, 'Петров', 46.89, '345 3893 fdjf')
(4, 'Сидоров', 67.89, '345 3893 fdjf  ooo')
```

Графический интерфейс

Для того чтобы создать (разработать) графический интерфейс (GUI) к своей программе нужно воспользоваться специальной библиотекой. GUI-библиотека содержит набор виджетов. Виджет – это графических элементов управления: кнопка, ползунок, окно и т.д.

GUI-библиотеки:
Tkinter
PySimpleGUI
CEF Python
Dabo
Kivy
Pyforms.
PyQt
PyGUI
libavg
PyGTK | PyGObject
wxPython



GUI-библиотеки

Библиотека **Tkinter** установлена в Python в качестве стандартного модуля, поэтому нам не нужно устанавливать что-либо для его использования.

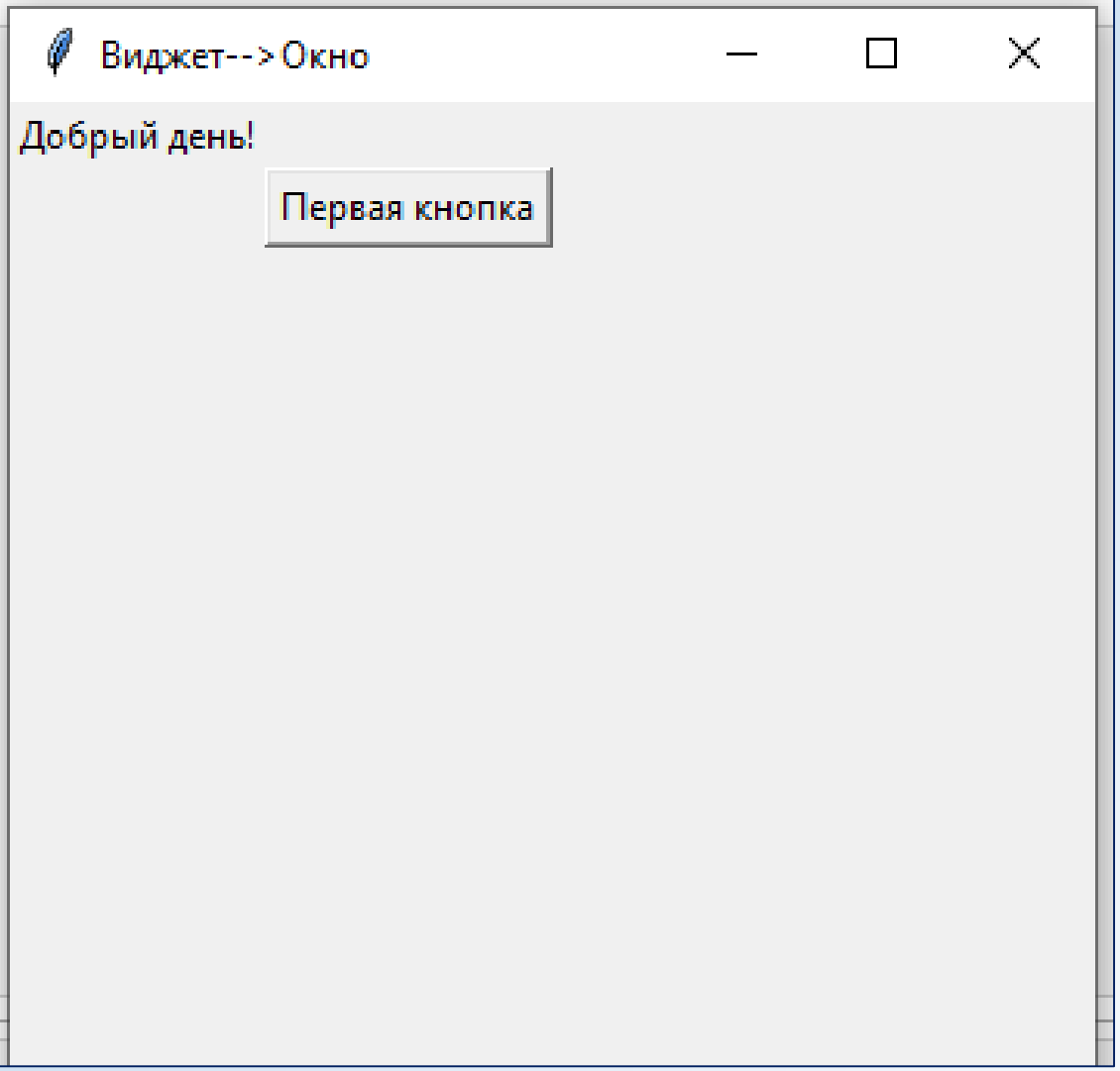
PyQt (PyQt5) реализован в виде набора python-модулей. Эта библиотека имеет более 620 классов и 6000 функций и методов. Это мультиплатформенная библиотека, которая работает на всех основных операционных системах, в том числе Unix, Windows и Mac OS.

PySimpleGUI – простая в использовании, но с достаточным набором возможностей для построения «незамысловатого» графического интерфейса к программе.

GUI-библиотека Tkinter

```
from tkinter import *
from tkinter.ttk import Combobox

# Создаём объект Окно
window = Tk()
window.title('Виджет-->Окно')
# размеры
window.geometry('350x350')
# подпись
label1 = Label(window, text="Добрый день!")
# положение на окне
label1.grid(column=0, row=0)
# кнопка button
knopka1 = Button(window, text="Первая кнопка")
knopka1.grid(column=3, row=1)
window.mainloop()
```



GUI-библиотека Tkinter

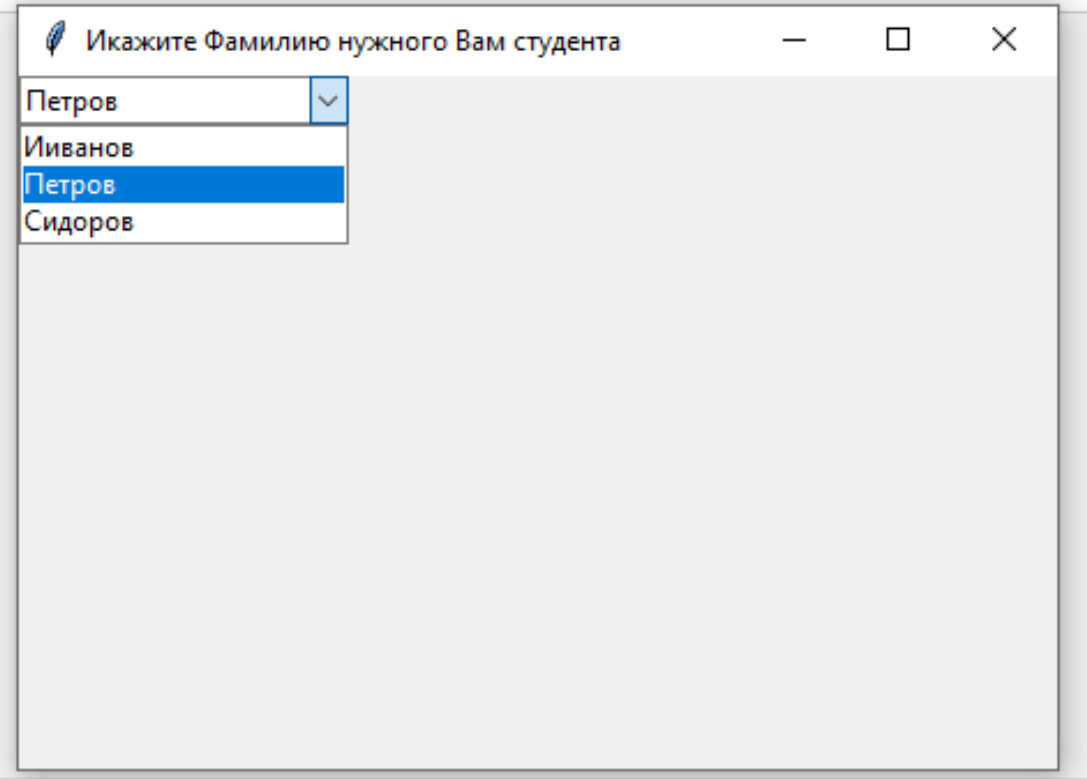
```
# получаем весь результат
cursor.execute("SELECT * FROM students")
results = cursor.fetchall()
print(*results, sep='\n')
```

```
(1, 'Иванов', 56.89, '345 3893 fdjf')
(3, 'Петров', 46.89, '345 3893 fdjf')
(4, 'Сидоров', 67.89, '345 3893 fdjf  ooo')
```

```
from tkinter import *
from tkinter.ttk import Combobox

# Создаём объект Окно
window = Tk()
window.title('Икажите Фамилию нужного Вам студента')
window.geometry('450x300')
combo = Combobox(window)
#Получаем Фамилии студентов и баллы
cursor.execute("SELECT name, balls FROM students")
results = cursor.fetchall()
names_stud = [x[0] for x in results]

combo['values'] = names_stud
combo.current(1)  # установите вариант по умолчанию
combo.grid(column=0, row=0)
window.mainloop()
```



GUI-библиотека Tkinter

In [62]:

```
# Создаём объект Окно
window1 = Tk()
window1.title('Икажите Фамилию нужного Вам студента')
window1.geometry('450x300')

# подпись
label1 = Label(window1, text="Здесь будут баллы студента", font=("Arial", 20))
label1.grid(column=0, row=0)

# combobox
combo = Combobox(window1)

#Получаем Фамилии студентов и баллы
cursor.execute("SELECT name, balls FROM students")
results = cursor.fetchall()
names_stud = [x[0] for x in results]

#заносим фамилии в combobox
combo['values'] = names_stud
combo.current(1)  # вариант по умолчанию
combo.grid(column=0, row=1)

# кнопка button
knopka1 = Button(window1, text="Посмотреть баллы", command=selected)
knopka1.grid(column=0, row=2)

window.mainloop()
```

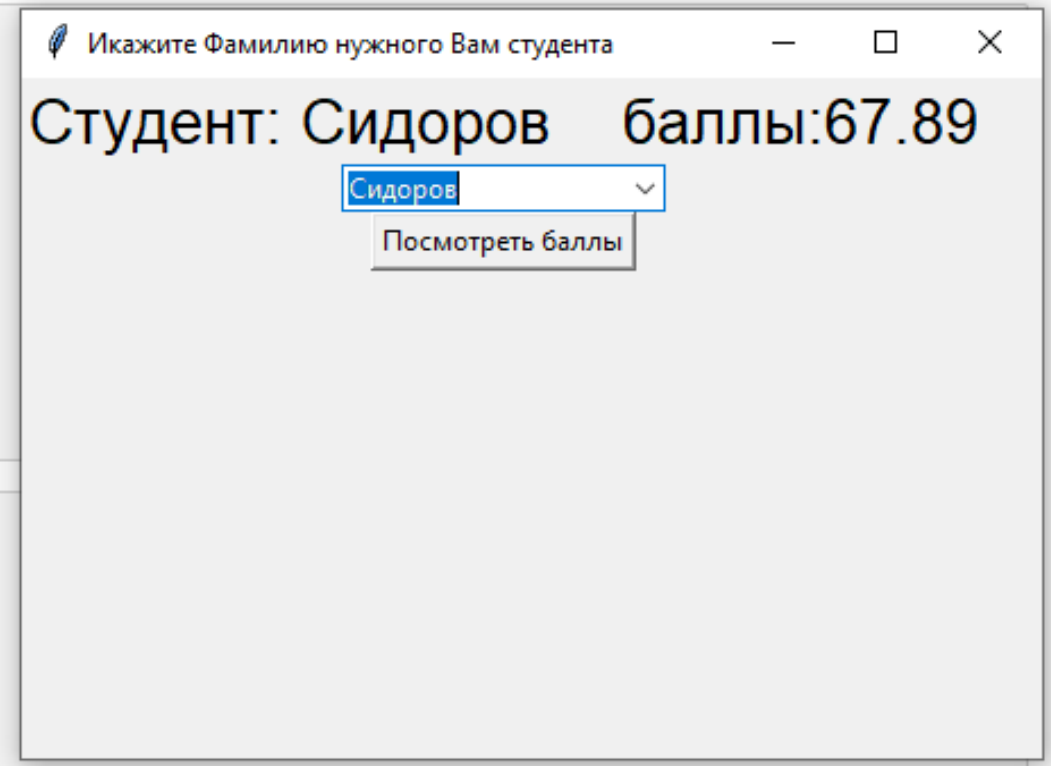
GUI-библиотека Tkinter

```
# функция для обработки события - нажатие на кнопку button
def selected():
    name = combo.get()
    balls = 0.0
    for x in results:
        if x[0]==name:
            balls = x[1]
            break
    label1.configure(text="Студент: "+str(name)+"    баллы:"+str(balls))
```

```


# Создаём объект Окно
window1 = Tk()
window1.title('Икажите Фамилию нужного Вам студента')
window1.geometry('450x300')

# подпись
label1 = Label(window1, text="Здесь будут баллы студента", font=("Arial", 20))
```



Web-frameworks

Веб-приложение состоит из двух частей: клиентской и серверной. Клиентская часть: HTML, CSS и Javascript. Реализация возможностей для представления информации пользователям.

Серверная часть: описывает бизнес-логику приложения. Веб-разработку серверной части можно выполнить на многих языках, таких как Java, .Net, PHP, Ruby, Javascript. Но, разработка серверной части без фреймворка очень ресурсозатратно, поэтому фреймворки и закрепились в качестве основы веб-разработки.



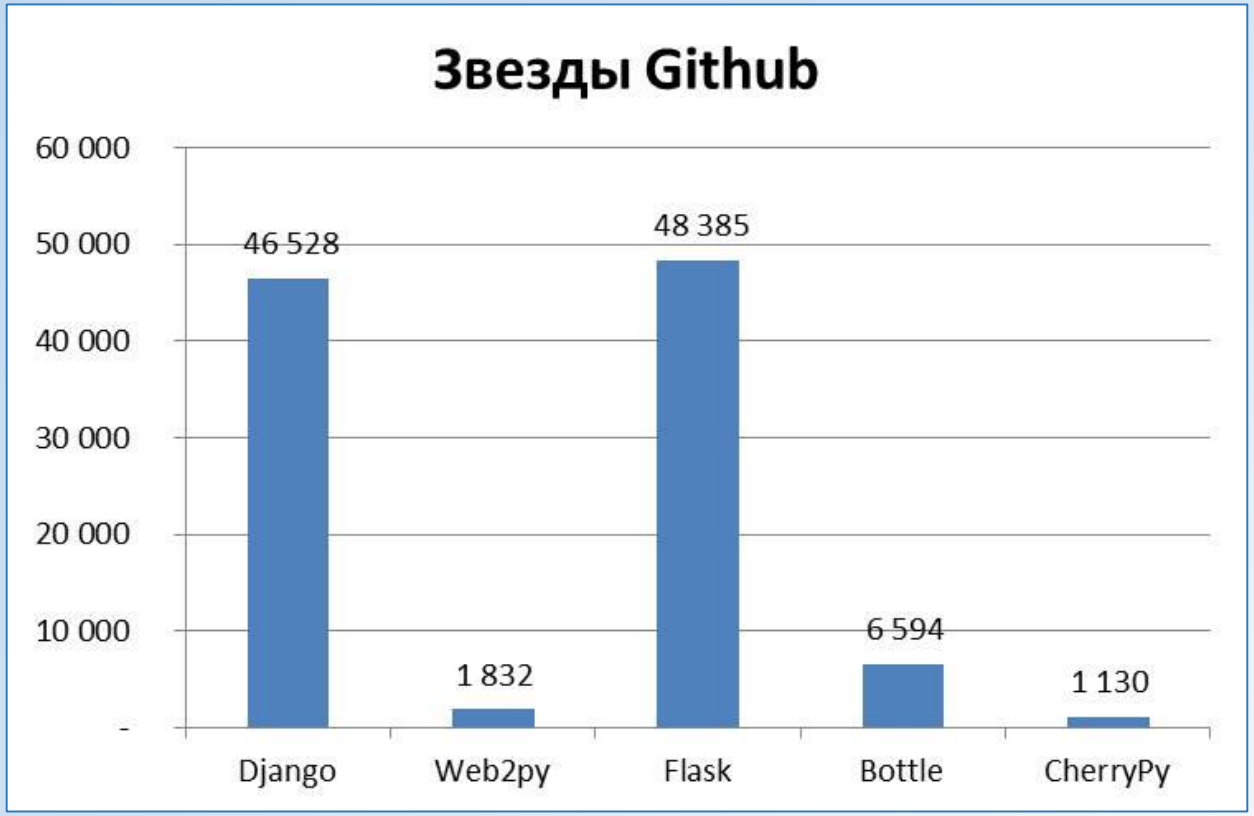
Web-frameworks

Django – бесплатный фреймворк полного стек, с открытым исходным кодом. Согласно документации: высокая скорость работы, надежность выполнения всех этапов реализации, защищенность данных, масштабируемость создаваемых разработок.

Web2py – фреймворк полного стека для быстрой, масштабируемой, безопасной разработки портативных веб-приложений с управлением базами данных.

Flask – микро-фреймворк с базовыми возможностями веб-разработки.

Bottle – простой и легкий мини-фреймворк. Представляет собой один файловый модуль и не имеет никаких зависимостей, кроме стандартной библиотеки Python



Web-framework Flask: создание и выполнение проекта

Установим Flask:

```
pip install flask
```

Создадим папку проекта (ОСНОВНАЯ ПАПКА ПРОЕКТА), а в ней основной файл с именем ***app.py***, в котором пропишем базовый код для приложения:

```
# импортируем Flask
from flask import Flask, render_template

# создаём приложение
app = Flask(__name__)

# Файл app.py отвечает и за маршрутизацию внутри проекта, и за доступ к функциям библиотеки Flask
#@app.route - это декоратор, который превращает стандартную функцию Python в функцию для фреймворка. Эта функция конвертирует результат обычной функции в HTTP- ответ, который затем отображается веб-браузером (HTTP- клиентом).

@app.route('/')
def index():
   return render_template('index.html')
#При вызове главной страницы проекта будет загружаться файл index.html, но его необходимо создать в каталоге templates.
```

Web-framework Flask : создание и выполнение проекта

Наполнение проекта:

В папке **templates** создадим файл **index.html**.

Желательно сначала создать файл-шаблон **main_template.html**, из которого все страницы проекта будут наследовать одинаковый дизайн.

В папке **static/css** следует создать стилевой файл: **main.scc**.

Запуск проекта:
Из основной папки проекта (там где находиться файл app.py) команда:
python.exe -m flask run

Выполнение проекта (тестирование):
В браузере следует указать адрес: http://127.0.0.1:5000

app.py

```python
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def index():  # put application's code here
    return render_template('index.html')
```

static

css

mail.scc

templates

main_template.html

```html
<!doctype html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, user-scalable=no, initial-scale=1.0, maximum-scale=1.0, minimum-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
     <!-- подключаем стилевой файл-->
    <link rel="stylesheet" href="./static/css/main.css">
    <title>Изучение фкеймворка Flask</title>
</head>
<body>
  <!-- в этот контейнер из других html-файлов будут вставляться фрагменты кода-->
  {% block body %}{% endblock %}
</body>
</html>
```

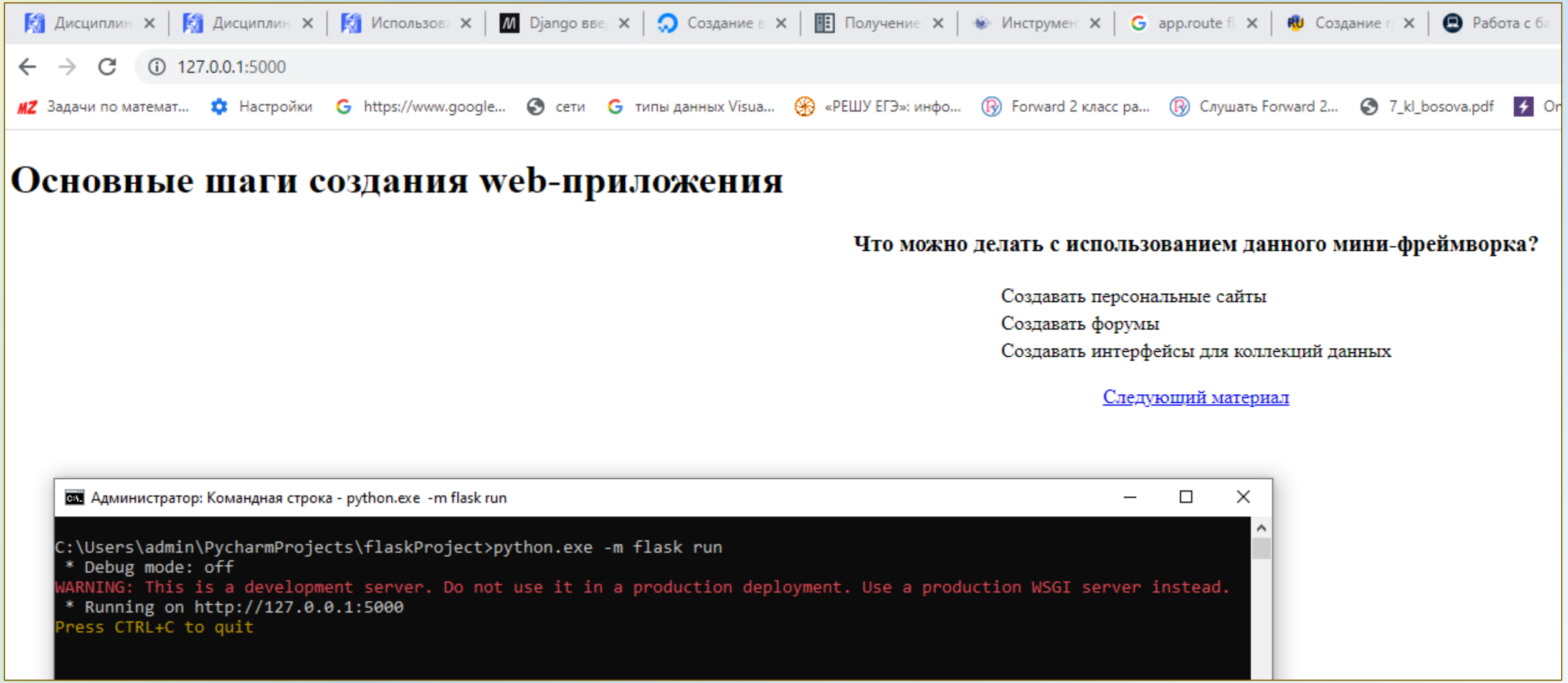
index.html

```html
 <!-- переносим настройки из этого файла-->
{% extends 'main_template.html' %}

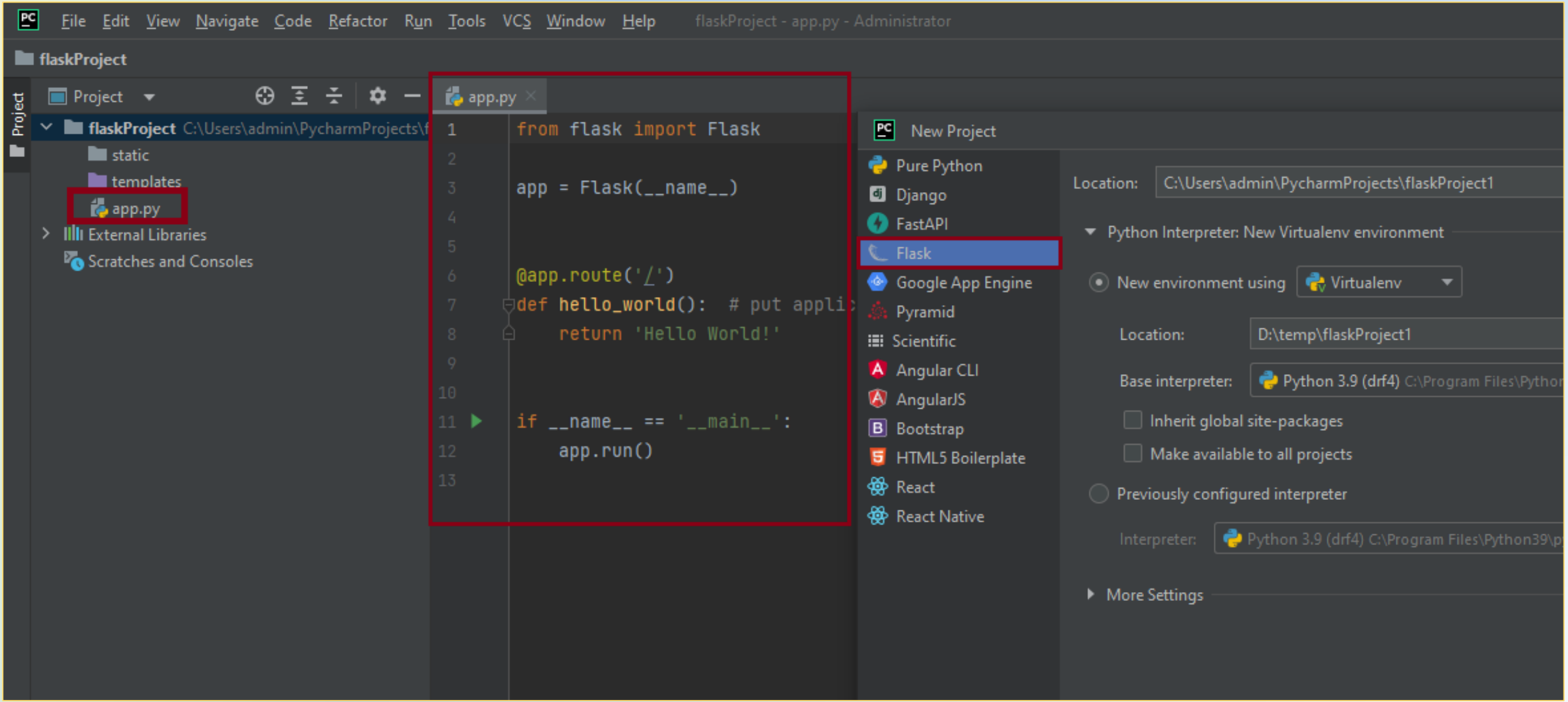
{% block body %}
    <div class="title">
        <h1>Основные шаги создания web-приложения</h1>
    </div>
    <h3 align="center">Что можно делать с использованием данного мини-фреймворка?</h3>
<div align="center">
    <table id="instruction">
            <tr id="row">
            <td>Создавать персональные сайты</td> </tr>
            <tr id="row">
            <td>Создавать форумы</td> </tr>
            <tr id="row">
            <td>Создавать интерфейсы для коллекций данных</td> </tr>
    </table>
</div>
<div align="center">
    <p><a class="btn" href="/lines" target="_blank">Следующий материал</a></p>
</div>

{% endblock %}
```

Web-framework Flask : создание и выполнение проекта

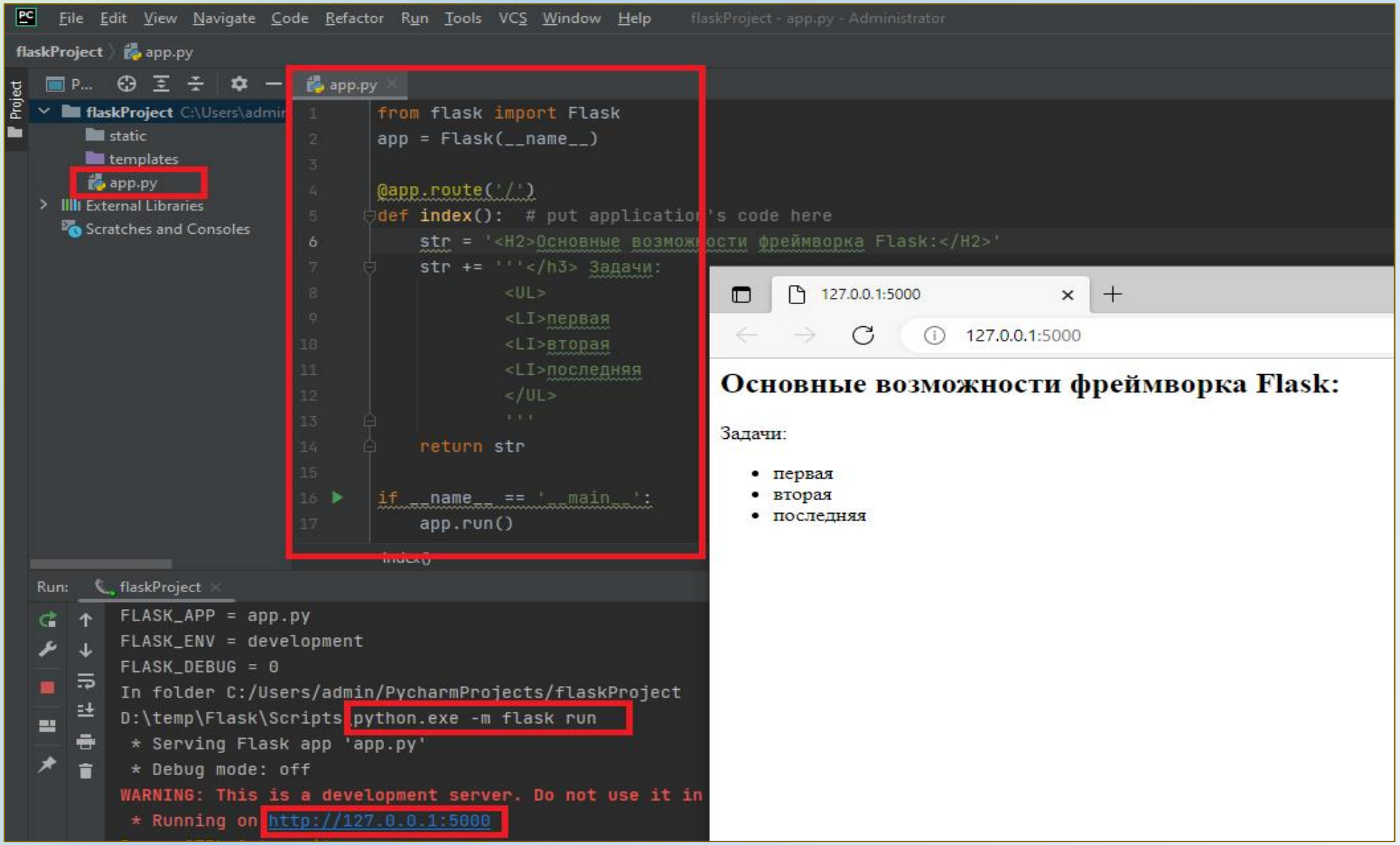


Web-framework Flask: создание проекта в PyCharm



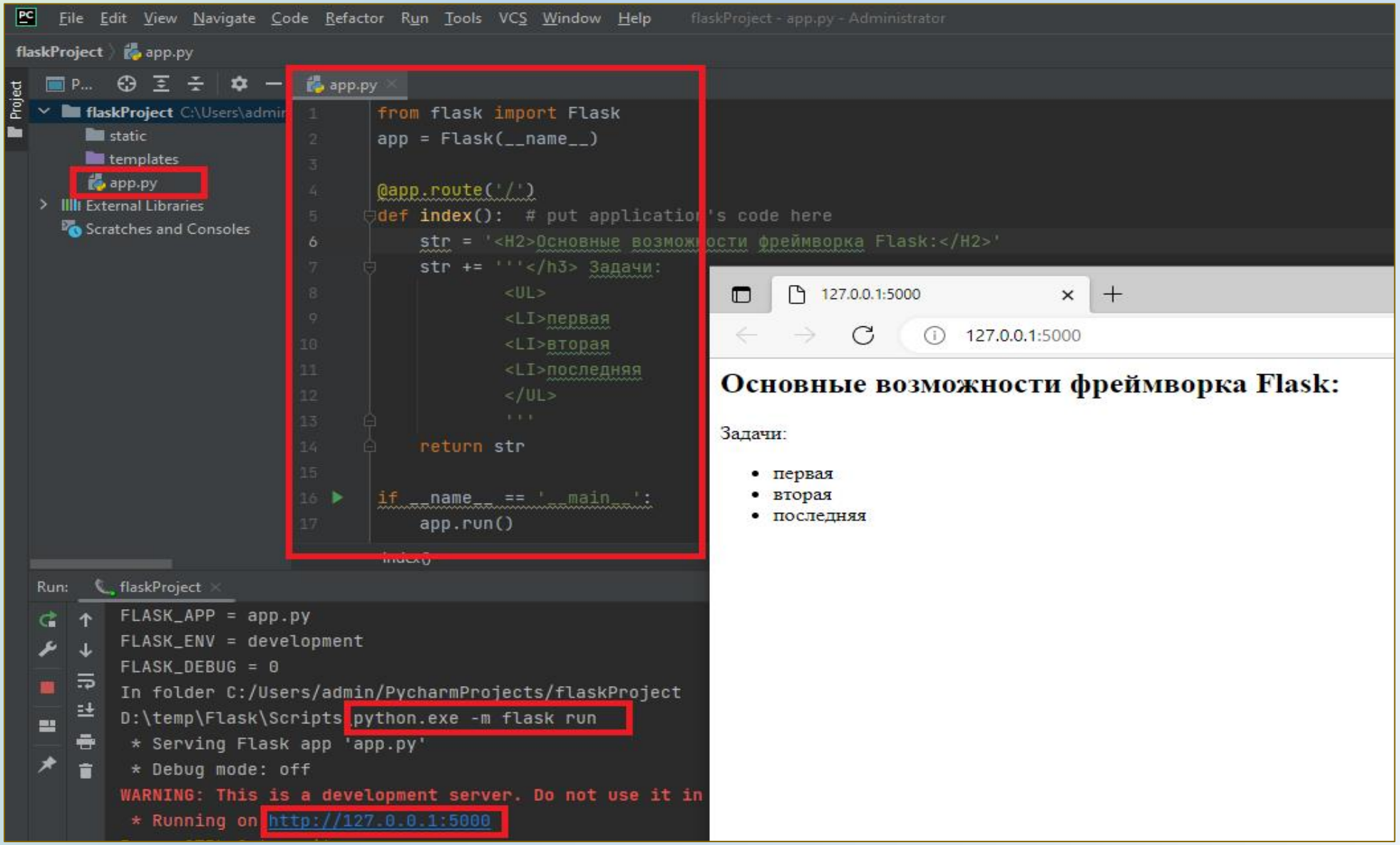
Создание проекта на основе Flask в PyCharm (Flask предварительно установлен)

Web-framework Flask : создание проекта в PyCharm



Выполнение проекта на основе Flask в PyCharm

Web-framework Flask : создание проекта в PyCharm



Выполнение проекта на основе Flask в PyCharm

Web-framework Flask : создание проекта в PyCharm

Далее следует загрузить и посмотреть видео:
https://video.asu.ru/media/flask_8lmp4

Выполнение проекта на основе Flask в PyCharm